

GRDR: A Semantic-ID Index and Recall–Rerank Query Engine for Scalable Text-to-Video Retrieval

Zecheng Zhao
The University of Queensland
Brisbane, Australia
uqzzha35@uq.edu.au

Zhi Chen
The University of Southern
Queensland
Toowoomba, Australia
zhi.chen@unisq.edu.au

Zi Huang
The University of Queensland
Brisbane, Australia
huang@itee.uq.edu.au

Shazia Sadiq
The University of Queensland
Brisbane, Australia
shazia@eecs.uq.edu.au

Tong Chen*
The University of Queensland
Brisbane, Australia
tong.chen@uq.edu.au

ABSTRACT

Large-scale text-to-video retrieval (TVR) needs a candidate generator with low storage cost, low query latency, and compatibility with accurate reranking. Dense dual-encoder retrieval is effective, but standard exhaustive serving stores high-dimensional video embeddings and scores queries against the full corpus. As a result, both index size and query latency grow linearly with corpus size. This makes dense retrieval a poor stage-1 recall mechanism at scale. Prior generative retrieval (GR) replaces dense embeddings with short discrete keys and avoids exhaustive full-corpus scoring. However, as a stage-1 candidate generator, it often misses videos that satisfy multiple query intents and learns keys that are hard to decode from text queries. This happens because key assignment is learned only from video features, without retrieval-aware text supervision. We present GRDR, a multi-view semantic-ID index and recall-rerank pipeline for scalable TVR. (i) Offline, GRDR assigns multiple short discrete keys to each video to expose parallel semantic access paths. It jointly constructs these keys with the query decoder under a shared vocabulary to improve query-to-key alignment. (ii) Online, GRDR performs prefix-pruned decoding over valid keys to build a compact candidate set, then reranks the candidates with a dense video-text matcher. Across four TVR benchmarks, including full-corpus evaluation, GRDR approaches strong dense baselines in retrieval quality. It also reduces index storage by more than an order of magnitude and lowers query latency by up to 300× as corpus size grows.

PVLDB Reference Format:

Zecheng Zhao, Zhi Chen, Zi Huang, Shazia Sadiq, and Tong Chen. GRDR: A Semantic-ID Index and Recall–Rerank Query Engine for Scalable Text-to-Video Retrieval. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

*Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

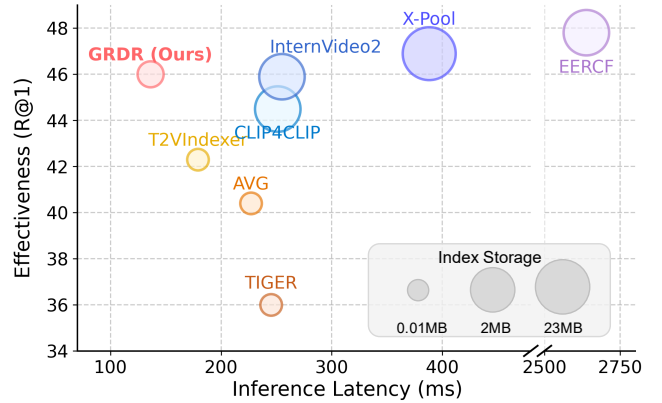


Figure 1: Effectiveness vs. Efficiency on MSRVT-1k test set [49]. Bubble size indicates the index storage required for cached video features. GRDR (Ours) achieves the optimal balance (positioned at the top-left).

The source code, data, and/or other artifacts have been made available at <https://github.com/JasonCodeMaker/GRDR>.

1 INTRODUCTION

Modern video platforms serve text queries over very large video corpora under tight latency budgets. Given a natural-language query, a text-to-video retrieval [20, 29, 34] system ranks videos in the corpus by their semantic relevance to the query and returns the top matches to the user. A production TVR service must simultaneously satisfy four requirements: (i) return results relevant to the query, (ii) respond with low latency, (iii) keep per-video index storage tractable as the corpus grows, and (iv) preserve the fine-grained matching quality users expect. These requirements cannot be satisfied by a single monolithic retriever: *accurate cross-modal matching is computationally heavy, and exhaustive scoring over a corpus of this scale is infeasible at query time*. Large-scale TVR systems therefore adopt a two-stage architecture [5, 6, 14, 43, 52]. Stage 1 is a fast candidate generator that narrows the corpus to a small pool of likely matches. Stage 2 is a stronger but more expensive reranker that scores this pool and produces the final ordering. In this design, stage 1 dictates both serving cost and recall quality. Its representation size governs memory, its query-time work governs latency, and any

target video it omits cannot be recovered downstream. Building a compact, fast, and high-coverage stage-1 candidate generator is therefore the central system problem for repository-scale TVR.

A common stage-1 mechanism in TVR is dense dual-encoder retrieval [10, 34, 46, 50], which maps videos and queries into a shared embedding space and retrieves candidates by vector similarity. This design supports diverse natural-language queries and satisfies all four requirements on small corpora. However, at repository scale, two of these requirements break down. (i) Index storage no longer stays tractable: each video is stored as a high-dimensional embedding, and frame-level variants store several embeddings per video, so the index footprint grows linearly with corpus size N . (ii) Query latency no longer stays low: the standard exhaustive pipeline scores the query against every stored embedding, so query-time work also grows linearly with N . Approximate nearest-neighbor (ANN) indexes (e.g., HNSW [35], IVF-PQ [15]) make the search sublinear and can compress stored vectors, but they preserve the same underlying access mechanism. The storage footprint therefore still scales with the corpus, and the query path remains corpus-dependent. Dense retrieval is thus a poor fit for stage-1 recall at scale, though it remains well suited as a stage-2 reranker over a small candidate pool.

Within this two-stage architecture, stage 1 and stage 2 serve fundamentally different system roles. Stage 1 must maximize candidate coverage under tight storage and latency budgets, whereas stage 2 optimizes fine-grained ranking quality over an already-bounded candidate pool. Because the reranker operates only on a small candidate set, its per-query cost is independent of corpus size. Even computationally expensive cross-modal models can therefore be deployed at stage 2 without inflating end-to-end latency. The implication for repository-scale TVR is that the *central system bottleneck is not fine-grained matching itself, but the stage-1 index that feeds it*. A target video missed at stage 1 cannot be recovered by any reranker, however accurate. This paper focuses on stage 1. Our work does not replace fine-grained cross-modal matching; it replaces the stage-1 candidate generator and assumes a strong dense reranker operates downstream on the compact pool it produces.

A viable stage-1 index needs an access mechanism whose storage and query-time cost do not scale with the corpus. Generative retrieval (GR) offers exactly that. Each video is assigned a short sequence of discrete tokens, called a *semantic ID*, drawn from a small learned vocabulary. Unlike the opaque keys used in classical keyed indexes, semantic IDs are learned jointly with the corpus and with a query-side decoder, so that semantically similar videos share prefixes and the mapping from a natural-language query to its target key can itself be learned. The corpus is therefore reduced, at the index level, to a trie whose paths enumerate the semantic IDs present in the index, plus a key-to-video lookup table. No per-video vector is stored. A query is resolved by traversing this index rather than searching it: a small autoregressive decoder consumes the query text and emits tokens one at a time, constrained at each step to valid paths in the trie, and a beam search returns a bounded set of semantic IDs that are resolved to videos through the lookup table. The returned candidate set size is controlled by the beam width, not by the number of videos in the corpus. Three index-level properties follow: (i) The stored state per video is a short token sequence rather than a high-dimensional embedding, so the index

footprint is dominated by the trie and shrinks by more than an order of magnitude in practice. (ii) Query-time work is determined by decoder size, key length, and beam width, all independent of corpus size N , so the online path performs no scan or search over per-video state. (iii) The key space is organized by construction: shared prefixes group semantically related videos, giving the index structured access paths rather than an unordered set of vectors. These properties make GR a structurally promising stage-1 index for repository-scale TVR.

However, existing GR methods construct semantic IDs in ways that undermine both stage-1 properties: high recall coverage and reliable query-to-key reachability. **Insufficient access-path coverage.** Existing GR methods assign a single semantic ID to each video, so the index exposes exactly one access path per video. Videos are inherently polysemous. For example, a cooking video is a legitimate target for queries about the dish, the technique, the ingredient, or the kitchen, but a single stored ID can encode only one of these facets. Queries whose intent aligns with any other facet cannot reach the target through the index at all, and a video missed at this stage cannot be recovered by any reranker downstream. The system consequence is a hard ceiling on stage-1 recall that scales with the diversity of real query intent. **Low query-to-key reachability.** Existing GR pipelines construct semantic IDs from video-side features alone and fit the query decoder afterward against the already-fixed IDs. The two halves of the index are therefore optimized in disjoint spaces: the keys are visually coherent but were never shaped by the retrieval objective, and the decoder is left to map text onto a key space it had no hand in defining. The system consequence is that even well-formed queries decode to keys that resolve to irrelevant videos, contaminating the candidate pool that stage 2 receives. Together, prior GR therefore wins on storage and latency but loses on coverage and addressability. These are the two axes a stage-1 index cannot afford to lose on.

We present GRDR, a multi-view semantic-ID index with a recall-rerank query pipeline for repository-scale TVR. GRDR inherits GR’s compact storage and corpus-independent query cost, and adds three design choices. Together, they address the two construction failures of prior GR and deliver the fine-grained matching quality that a stage-1 index alone cannot provide. (i) To resolve insufficient access-path coverage, GRDR assigns each video multiple semantic IDs instead of one, exposing its polysemous facets as parallel access paths. (ii) To close the query-to-key reachability gap, GRDR jointly trains the semantic IDs and the query decoder under a shared vocabulary, so the stored keys are shaped by the same objective that later traverses them. (iii) To recover fine-grained matching quality, GRDR reranks the decoded candidate pool with an off-the-shelf dense cross-modal model, whose cost scales with the pool size rather than the corpus. Figure 2(c) contrasts this design with dense retrieval and prior GR.

Figure 3 shows how GRDR operates in two phases. (i) **Offline**, each video in the corpus is encoded into several short semantic IDs by a multi-view encoder, and all IDs are organized into a trie that forms the index. Key assignment and query decoding are trained together over a shared vocabulary, so the stored keys are shaped by the same objective that will later retrieve them. (ii) **Online**, the decoder reads the query and walks the trie to produce a small set of candidate IDs, which are mapped to videos through a lookup table.

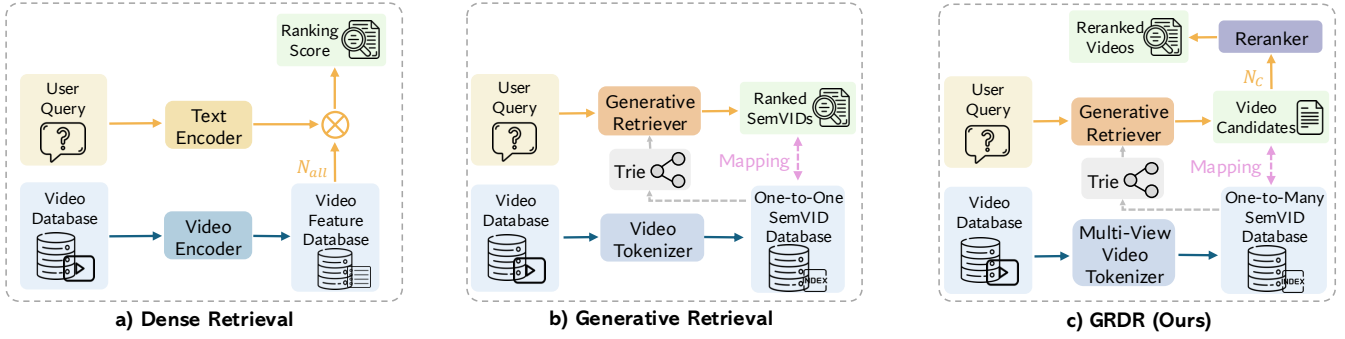


Figure 2: Overview of Text-to-Video Retrieval Paradigms. (a) Dense Retrieval: Decouples modalities into dual-encoders, relying on exhaustive similarity search over high-dimensional embeddings. (b) Generative Retrieval: Reformulates retrieval as sequence-to-sequence generation, constrained by a rigid one-to-one video-to-identifier mapping. (c) GRDR (Ours): leverage a Recall-then-Rerank paradigm using multi-view (one-to-many) tokenization to capture diverse video semantics.

A dense reranker then scores this bounded pool and returns the final ranking. The online path touches no per-video embeddings and runs no similarity search over the corpus. All corpus-scale work is absorbed offline.

We summarize the contributions of this paper as follows.

- **Stage-1 candidate generation as a generative indexing problem.** We identify the stage-1 index as the central bottleneck for repository-scale TVR and reformulate its construction as a generative indexing problem. GRDR serves as a drop-in stage-1 replacement that pairs with any dense reranker, absorbing corpus-scale work offline.
- **A multi-access-path, retrieval-aware index.** GRDR assigns multiple semantic IDs per video to address insufficient access-path coverage, and jointly trains key assignment with the query decoder under a shared vocabulary to close the query-to-key reachability gap.
- **Full-corpus evaluation of quality, storage, and latency.** On four TVR benchmarks under both inductive and full-corpus settings, GRDR matches strong dense retrievers in quality while cutting index storage by over an order of magnitude and query latency by up to 300× as the corpus grows.

2 BACKGROUND AND PROBLEM DEFINITION

This section formalizes the two-stage retrieval pipeline that GRDR targets (Section 2.1), characterizes the stage-1 index design space along storage and access dimensions (Section 2.2), and states the stage-1 design goal as a constrained optimization problem (Section 2.3).

2.1 Two-Stage TVR Pipeline

We model repository-scale TVR as a two-stage serving pipeline over a video repository V . Given a text query q , stage 1 consults an index I to produce a bounded candidate set, and stage 2 reranks that set to produce the final order. Let $G(\cdot)$ denote the stage-1 candidate generator and $R(\cdot)$ the stage-2 reranker. The stage-1 output is

$$C_B(q) = G(q; I), \quad |C_B(q)| \leq B, \quad (1)$$

where B is the candidate budget. The final ranking over $C_B(q)$ is

$$\pi(q) = R(q, C_B(q)). \quad (2)$$

The two stages play complementary system roles. Stage 1 determines the stored state of the serving index, the query-time work performed before reranking, and the recall coverage delivered downstream. Any relevant video absent from $C_B(q)$ is unrecoverable, however accurate R is. Stage 2, in contrast, operates on a pool whose size is bounded by B and is therefore independent of $|V|$; even computationally expensive cross-modal scorers can be deployed at this stage without inflating end-to-end latency. The implication is a clear division of objectives: stage 1 must maximize candidate coverage under tight storage and latency budgets, while stage 2 optimizes fine-grained ranking quality over an already-bounded pool. This paper targets stage 1.

2.2 Stage-1 Index Design Space

A stage-1 TVR index is characterized by two design choices: (i) the per-video state stored in the index, and (ii) the access mechanism used to materialize candidates at query time. Existing TVR systems fall into two categories along these axes.

Dense retrieval indexes Each video $v \in V$ is stored as one or more high-dimensional embeddings $E(v) \in \mathbb{R}^d$. At query time, the query is encoded into the same space and candidates are produced by similarity search:

$$C_B(q) = \text{Top-}B_{v \in V} \text{sim}(E(q), E(v)). \quad (3)$$

Per-video stored state is $\Theta(d)$ floats, and the per-query work scales linearly with $|V|$ unless approximated; ANN structures (e.g., HNSW [35], IVF-PQ [15]) reduce the search cost but preserve the same access mechanism over per-video vectors.

Semantic-ID indexes Each video is represented by one or more short discrete keys drawn from a learned vocabulary, organized into a prefix structure T (a trie). Let $\mathcal{S}(T)$ denote the set of valid keys in T , and let $M : \mathcal{S}(T) \rightarrow 2^V$ denote the key-to-video lookup table. At query time, an autoregressive decoder constrained to T produces a bounded set of L candidate keys via beam search:

$$\hat{S}_L(q) = \text{Top-}L_{s \in \mathcal{S}(T)} p(s | q). \quad (4)$$

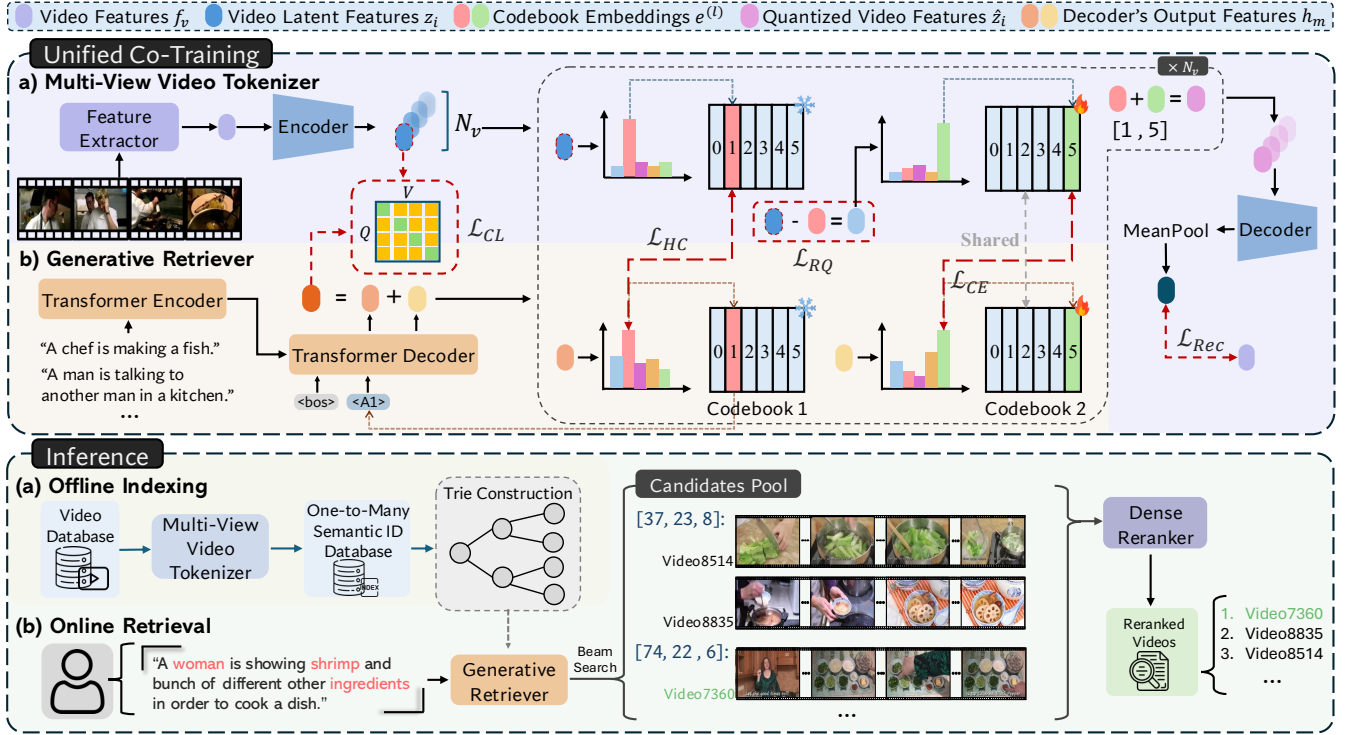


Figure 3: Overview of the GRDR framework. Training: (a) The multi-view video tokenizer encodes videos into discrete semantic IDs. It employs cross-modal alignment ($\mathcal{L}_{CL}$) to align video latent features z_i with the retriever’s cumulative decoder features $h_m^{(m)}$ via contrastive learning. The video latent features z_i then passed to residual quantization ($\mathcal{L}_{RQ}$), followed by a reconstruction decoder ($\mathcal{L}_{Rec}$) to prevent semantic collapse. (b) The generative retriever is jointly optimized with the tokenizer via a shared codebook. Its decoder predicts codes using cosine similarity, trained with cross-entropy loss ($\mathcal{L}_{CE}$). We employ progressive training, where Hierarchical Consistency ($\mathcal{L}_{HC}$) loss is used to prevent code drift in previous layers. The inference pipeline follows a Generative Recall, Dense Reranking paradigm: (a) Offline Indexing: Videos are tokenized into compact IDs to construct a prefix Trie for constrained decoding. (b) Online Retrieval: The retriever generates candidate Semantic IDs via beam search.

The keys are then resolved into candidate videos through M :

$$\tilde{C}(q) = \bigcup_{s \in \hat{S}_L(q)} M(s). \quad (5)$$

The stage-1 output is the deduplicated set

$$C_B(q) \subseteq \tilde{C}(q), \quad |C_B(q)| \leq B. \quad (6)$$

Per-video stored state is a short token sequence rather than a d -dimensional vector, and per-query work depends on decoder size, key length, and beam width L —all independent of $|V|$.

2.3 Problem Statement

A stage-1 index is judged by the candidate set it delivers under serving cost, not by fine-grained ranking accuracy in isolation. For a query q with target video $v^*(q)$, an effective stage-1 design simultaneously pursues three desiderata: (i) **high candidate recall**, maximizing $\Pr[v^*(q) \in C_B(q)]$, the probability that the target video is contained in the bounded candidate set; (ii) **compact index storage**, reducing the per-video state $\text{Storage}(I)$ retained in the serving index; and (iii) **low query latency**, reducing the per-query work $\text{Latency}(G; I)$ performed before reranking. These objectives are inherently in tension: richer indexes improve recall but inflate

storage and latency, while aggressive compression or shallower querying lowers cost at the expense of recall. Fine-grained ranking is delegated to stage 2 and is therefore outside this objective.

This formulation exposes two index-level properties that any semantic-ID stage-1 design must satisfy: (i) **access-path coverage**, where a relevant video must be reachable through at least one stored key consistent with the query intent; and (ii) **query-to-key reachability**, where the query-side decoder must assign high probability to keys whose lookup under M resolves to relevant videos. A failure on either axis directly reduces $\Pr[v^*(q) \in C_B(q)]$ and cannot be repaired by any downstream reranker.

3 SYSTEM OVERVIEW

GRDR instantiates the stage-1 candidate generator $G(q; I)$ from § 2.3 as a semantic-ID index served by an autoregressive decoder. As shown in Figure 3, the system comprises *three persisted data structures* and *two pipelines that operate on them*. The **offline pipeline** constructs the data structures: it assigns each video multiple short semantic-ID keys, jointly trains these keys with a query decoder over a shared vocabulary, and materializes the serving index. The **online pipeline** queries them: the decoder walks the index under

prefix constraints, resolves decoded keys to videos, deduplicates, and hands the bounded candidate pool to a dense reranker. This decomposition confines all corpus-scale computation to the offline pipeline. A query processing reduces to prefix-constrained decoding and a bounded number of key lookups, regardless of $|V|$. § 3.1 describes the three data structures; § 3.2 describes the offline pipeline that builds them; § 3.3 describes the online pipeline that queries them.

3.1 Index Structure and Stored State

After offline construction, GRDR persists exactly three objects, which together instantiate the stage-1 index I : **(i) A query decoder** D . A small autoregressive model that maps a text query to a distribution over semantic-ID prefixes. Its parameters are fixed after training and do not grow with $|V|$. **(ii) A prefix trie** T over the valid semantic-ID keys in the repository. Each root-to-leaf path is a key in $\mathcal{S}(T)$, and sibling branches share prefixes. T compactly represents the admissible decoding space and supports constant-time transitions per decoding step. **(iii) A key-to-video mapping** $M : \mathcal{S}(T) \rightarrow 2^V$, implemented as a hash table. M resolves each decoded key to the set of videos indexed under it. Because each video is assigned N_v keys and multiple videos may share a key, M is many-to-many.

None of $\{D, T, M\}$ stores per-video dense embeddings or vector-search structures. The per-video footprint is a short sequence of discrete tokens rather than a d -dimensional float vector, which yields an order-of-magnitude storage reduction over dense dual-encoder indexes. Stage-2 reranking is performed by an off-the-shelf dense model R that operates only on the bounded candidate pool produced by stage 1. Its parameters are persisted separately from $\{D, T, M\}$ and are not part of the stage-1 index under analysis.

3.2 Offline Indexing

The three objects $\{D, T, M\}$ are produced by a two-step pipeline over the video repository V . **Step 1: Joint key assignment and decoder training.** Rather than assigning a single key per video, GRDR assigns N_v short semantic-ID keys to each video, so that a polysemous video becomes reachable through N_v parallel paths in the index. Key assignment and query decoding share a single codebook and are optimized under one retrieval-aware objective: the keys produced offline are the same keys D learns to emit from text. This coupling directly targets the access-path coverage and query-to-key reachability requirements identified in § 2.3; § 4 details the training procedure. **Step 2: Index materialization.** The assigned keys are inserted into the prefix trie T , and the mapping M is populated with the resulting (key, video) entries. After this step, D , T , and M are persisted as the stage-1 serving state. All corpus-scale computation is absorbed into these two steps. Thus, the online pipeline described next touches none of it.

3.3 Online Query Path

Given a text query q at serving time, GRDR produces the stage-1 candidate set by consulting $\{D, T, M\}$ in four steps: **(i) Prefix-constrained decoding** (D, T). The query decoder D consumes q and emits semantic-ID tokens autoregressively. At each step,

token expansion is restricted to children that exist in T , so every completed key is guaranteed to be a valid index key. A beam search of width L returns the top- L keys $\hat{S}_L(q)$ (Eq. (4)). **(ii) Key-to-video resolution** M . Each decoded key $s \in \hat{S}_L(q)$ is resolved through M to its associated videos, forming the intermediate multi-set $\tilde{C}(q) = \bigcup_{s \in \hat{S}_L(q)} M(s)$ (Eq. (5)). **(iii) Deduplication.** Because each video maps to N_v keys and keys may collide across videos, $\tilde{C}(q)$ can contain duplicates. GRDR deduplicates by first occurrence to preserve the beam-induced ranking, yielding the bounded candidate set $C_B(q)$ (Eq. (6)). **(iv) Dense reranking** R . The reranker scores each pair (q, v) for $v \in C_B(q)$ and returns the final ranking $\pi(q)$.

Steps 1–3 constitute stage 1. Step 1’s cost depends only on decoder size, key length, and beam width L ; step 2 is a bounded number of hash-table lookups; step 3 is a bounded sort. None of these costs depend on $|V|$. Step 4 scores at most B pairs and is the only stage that performs fine-grained cross-modal matching.

4 SEMANTIC-ID INDEX CONSTRUCTION

This section instantiates Step 1 of the offline pipeline §3.2: the joint training procedure that produces the N_v semantic-ID keys assigned to each video and the query decoder D that later emits them. GRDR couples a multi-view tokenizer with the query decoder under a single retrieval-aware objective over a shared codebook. This design resolves the two failure modes identified in §2.3. For *insufficient access-path coverage*, §4.1 constructs a multi-view tokenizer that emits N_v parallel keys per video, each shaped by a distinct cluster of query intents. For *query-to-key reachability gap*, §4.2 jointly trains the tokenizer and D over a shared codebook, so the stored keys are shaped by the same objective that later traverses them.

4.1 Multi-View Key Assignment

Key assignment is performed by a multi-view tokenizer that maps each video v to N_v semantic-ID keys. A multi-view tokenizer is not obtained by simply replicating a single-view tokenizer N_v times. Without explicit supervision, the views collapse to near-identical representations; and any diversity that does emerge need not align with the facets real queries express. As shown in Figure 3(a), our tokenizer addresses both issues through three components: (i) a *query-guided multi-view encoder* that projects each video into N_v latent features, each shaped by a distinct cluster of query intents; (ii) a *residual quantizer* that discretizes each latent into a hierarchical code sequence; and (iii) a *reconstruction decoder* that prevents view collapse during quantization.

Query-Guided Multi-View Encoder. Given a raw video v with T sampled frames, we extract a shared video feature $\mathbf{f}_v$ using a pre-trained Vision-Language Model (VLM) backbone (InternVideo2 [46]). $\mathbf{f}_v$ is then fed to N_v independent sub-encoders $\{\phi_1, \phi_2, \dots, \phi_{N_v}\}$:

$$\mathbf{z}_i = \phi_i(\mathbf{f}_v), \quad i \in \{1, 2, \dots, N_v\}, \quad (7)$$

where $\mathbf{z}_i \in \mathbb{R}^{d_z}$. To ground the views in real query distributions rather than arbitrary decompositions, we cluster the queries associated with each video into $K = N_v$ groups via k-means over their VLM [46] embeddings; sub-encoder ϕ_i is trained to align

with queries in cluster i . The alignment is enforced through a cross-modal contrastive loss $\mathcal{L}_{CL}$ between $\mathbf{z}_i$ and the corresponding query features extracted from the retriever’s decoder (Eq. 13).

Residual Quantization. Each view’s latent $\mathbf{z}_i$ is discretized into a sequence of M codes through Residual Quantization (RQ) [21]. RQ maintains M codebooks $\{C^{(m)}\}_{m=1}^M$, where each $C^{(m)} = \{\mathbf{e}_1^{(m)}, \dots, \mathbf{e}_K^{(m)}\}$ contains K learnable embeddings; the selected code indices form the semantic ID of the view. At each layer m , RQ first computes the residual between the latent and its accumulated approximation from previous layers:

$$\mathbf{r}_i^{(1)} = \mathbf{z}_i, \quad \mathbf{r}_i^{(m)} = \mathbf{z}_i - \sum_{l=1}^{m-1} \mathbf{e}_{c_i^{(l)}}^{(l)}, \quad m > 1. \quad (8)$$

It then selects the codebook entry with maximum cosine similarity to the residual:

$$c_i^{(m)} = \arg \max_{k \in \{1, \dots, K\}} \cos(\mathbf{r}_i^{(m)}, \mathbf{e}_k^{(m)}). \quad (9)$$

We adopt cosine similarity rather than L_2 distance because the resulting codebook is directly compatible with the decoder’s logit computation, enabling the joint optimization in §4.2. RQ is trained with two complementary terms:

$$\mathcal{L}_{RQ} = \underbrace{\|\hat{\mathbf{z}}_i - \text{sg}[\mathbf{z}_i]\|_2^2}_{\text{codebook loss}} + \beta \cdot \underbrace{\|\text{sg}[\hat{\mathbf{z}}_i] - \mathbf{z}_i\|_2^2}_{\text{commitment loss}}, \quad (10)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator and $\hat{\mathbf{z}}_i = \sum_m \mathbf{e}_{c_i^{(m)}}^{(m)}$ is the reconstructed quantized feature.

Reconstruction Decoder. To preserve video semantics and prevent the views from collapsing under quantization, a set of view-specific sub-decoders $\{\psi_i\}_{i=1}^{N_v}$ reconstructs each view independently:

$$\tilde{\mathbf{f}}_i = \psi_i(\hat{\mathbf{z}}_i), \quad i \in \{1, 2, \dots, N_v\}. \quad (11)$$

The final reconstructed feature $\tilde{\mathbf{f}}_v$ is the mean-pooled aggregate across views. We regularize the quantization with a cosine-distance reconstruction loss $\mathcal{L}_{Rec} = 1 - \cos(\mathbf{f}_v, \tilde{\mathbf{f}}_v)$ on top of $\mathcal{L}_{RQ}$.

4.2 Joint Key–Decoder Training

The keys produced by the tokenizer are the same keys D must emit from text. Training the two halves in separate stages leaves the keys visually coherent but shaped by no retrieval signal, so queries decode to keys whose lookup under M resolves to irrelevant videos (§2.3). Injecting a cross-modal alignment objective into tokenizer training (e.g., AVG [26]) only partially closes the gap, since D is still fit against keys it had no hand in shaping. We instead optimize both halves under a single objective, supported by two design choices: (i) a *shared codebook* that exposes the same code embeddings to both the tokenizer and D , supporting bidirectional gradient flow; and (ii) a *progressive training strategy* that optimizes one codebook layer at a time.

A naive instantiation of this coupling fails, because the two halves use incompatible code-selection geometries: the tokenizer assigns codes by L_2 nearest-neighbor, while D predicts codes via dot-product softmax. Aligning their distributions under this mismatch causes training collapse. We resolve this by reformulating both selections under cosine similarity over the same shared embeddings, as detailed below. Figure 3(b) summarizes the design.

Algorithm 1 Pseudocode of Joint Key–Decoder Training for GRDR

Input: $\mathbf{f}_v, q$, codebook layers M , size K

```

1: for  $m = 1$  to  $M$  do
2:   Freeze  $C^{(1:m-1)}$ 
3:   for epoch = 1 to align_epochs do  $\triangleright$  P1: Cross-modal alignment
4:     for each  $(v, q)$  in  $\mathcal{D}$  do
5:        $\mathbf{z}_i \leftarrow$  Eq. (7);  $\mathbf{h}^{(m)} \leftarrow \sum_{l=1}^m \mathbf{h}_l$ 
6:        $\mathcal{L} \leftarrow$  Eq. (13) +  $\mathbb{1}_{m>1} \cdot$  Eq. (14)
7:       Back-propagate and update
8:     end for
9:   end for
10:   $\mathbf{r}_i^{(m)} \leftarrow$  Eq. (8);  $C^{(m)} \leftarrow \text{KMeans}(\mathbf{r}^{(m)}, K)$   $\triangleright$  Initialize codebook
11:  for epoch = 1 to train_epochs do  $\triangleright$  P2: Unified co-training
12:    for each  $(v, q)$  in  $\mathcal{D}$  do
13:       $c_i^{(m)} \leftarrow$  Eq. (9);  $\tilde{\mathbf{f}}_v \leftarrow$  Eq. (11)  $\triangleright$  Video tokenizer
14:       $P(c_m | c_{<m}, q) \leftarrow$  Eq. (12)  $\triangleright$  Soft prediction from retriever
15:       $\mathcal{L}_{total} \leftarrow$  Eq. (16)
16:      Back-propagate and update
17:    end for
18:  end for
19: end for

```

Shared Codebook Mechanism. The codebook serves both as the quantization vocabulary in the tokenizer and as the generation vocabulary in D . To couple them end-to-end, we use the same $C^{(m)}$ at each layer m for both code selection and code generation. In D , token probabilities are computed by a softmax over cosine similarities between the decoder’s output feature $\mathbf{h}_m$ and $C^{(m)}$:

$$P(c_m | c_{<m}, q) = \text{softmax} \left(\cos(\mathbf{h}_m, C^{(m)}) / \tau \right). \quad (12)$$

Meanwhile, the tokenizer assigns codes by selecting the nearest entry in $C^{(m)}$ under cosine similarity (Eq. 9). Gradients from the retrieval loss therefore flow directly into the codebook, refining the tokenizer’s quantization boundaries; gradients from the reconstruction loss in turn shape the codes D learns to predict. Both modalities are optimized over a shared hypersphere.

The unified code-selection rule also enables direct video-text alignment before codebook learning. A contrastive objective $\mathcal{L}_{CL}$ aligns the tokenizer’s latent $\mathbf{z}_i$ with the retriever’s cumulative decoder feature $\mathbf{h}^{(m)} = \sum_{l=1}^m \mathbf{h}_l$, mirroring the residual addition structure of quantization:

$$\mathcal{L}_{CL} = -\log \frac{\exp(\cos(\mathbf{z}_i, \mathbf{h}^{(m)})/\tau)}{\sum_{j=1}^B \exp(\cos(\mathbf{z}_j, \mathbf{h}^{(m)})/\tau)}, \quad (13)$$

where τ is a learnable temperature. When optimizing layer $m > 1$, a hierarchical consistency loss $\mathcal{L}_{HC}$ is applied on both modalities to prevent the codes assigned in the previous $m-1$ layers from drifting:

$$\mathcal{L}_{HC} = -\sum_{l=1}^{m-1} [\log P(c_l | c_{<l}, q) + \log P(c_l | c_{<l}, v)]. \quad (14)$$

This loss preserves the prediction capability of both D and the tokenizer for codes from preceding layers.

Progressive Training Strategy. The hierarchical structure of RQ naturally motivates a progressive schedule. Training all M layers simultaneously destabilizes optimization and produces suboptimal code allocation, since later layers shift the residuals on which earlier

Algorithm 2 Query processing in GRDR

Input: query q , index $\{D, T, M\}$, semantic-ID length K , beam width L , candidate budget B , reranker R

```

1:  $\mathcal{B}_0 \leftarrow \{(\langle \text{bos} \rangle, 0)\} \triangleright (\text{prefix}, \text{log-probability})$ 
2: for  $m = 1$  to  $K$  do
3:    $\mathcal{B}_m \leftarrow \emptyset$ 
4:   for each  $(s_{<m}, \ell) \in \mathcal{B}_{m-1}$  do
5:      $\mathcal{V} \leftarrow \text{Children}_T(s_{<m}) \triangleright \text{trie-admissible tokens}$ 
6:      $\mathcal{B}_m \leftarrow \mathcal{B}_m \cup \{(s_{<m} \cdot v, \ell + \log P(v \mid s_{<m}, q)) : v \in \mathcal{V}\}$ 
7:   end for
8:    $\mathcal{B}_m \leftarrow \text{Top-}L(\mathcal{B}_m)$ 
9: end for
10:  $\hat{S}_L(q) \leftarrow [s : (s, \cdot) \in \mathcal{B}_K] \triangleright \text{ordered by beam score}$ 
11:  $\tilde{C}(q) \leftarrow []$ 
12: for each  $s \in \hat{S}_L(q)$  do
13:    $\tilde{C}(q) \leftarrow \tilde{C}(q) \parallel M(s)$ 
14: end for
15:  $C(q) \leftarrow \text{Dedup}(\tilde{C}(q)) \triangleright \text{preserve first occurrence}$ 
16:  $C_B(q) \leftarrow \text{Prefix}_B(C(q))$ 
17: return  $R(q, C_B(q)) = 0$ 

```

layers were defined. We instead train one codebook layer at a time (Algorithm 1). For each layer m , training is split into two phases. Phase 1 aligns both modalities in the shared embedding space, realizing the query-guided encoding of §4.1. Phase 2 initializes $C^{(m)}$ by k-means clustering on the residual features, providing semantically meaningful starting points, and then jointly optimizes both halves. D learns to emit the tokenizer-assigned code via cross-entropy:

$$\mathcal{L}_{CE} = -\log P(c_m | c_{<m}, q), \quad (15)$$

where c_m is the code assigned by the tokenizer at layer m . The full co-training objective combines all losses:

$$\mathcal{L}_{total} = \underbrace{\lambda_1 \mathcal{L}_{CE} + \lambda_2 \mathcal{L}_{HC}}_{\text{Generative Retriever}} + \underbrace{\lambda_3 \mathcal{L}_{RQ} + \lambda_4 \mathcal{L}_{Rec}}_{\text{Video Tokenizer}}, \quad (16)$$

where $\lambda_1, \dots, \lambda_4$ are weighting coefficients.

5 QUERY PROCESSING

This section instantiates the online pipeline in §3.3: the query-time procedure that maps a text query q to the bounded candidate pool $C_B(q)$ over the persisted stage-1 state $\{D, T, M\}$. GRDR realizes this procedure through three stage-1 operators: prefix-constrained key decoding, key-to-video lookup, and deduplication followed by reranker handoff. This design implements the stage-1 candidate generator $G(q; I)$ as a decode-and-lookup access path over the materialized semantic-ID index. Algorithm 2 summarizes the composition of these operators. Specifically, §5.1 maps q to an ordered list $\hat{S}_L(q)$ of valid semantic-ID keys, §5.2 resolves $\hat{S}_L(q)$ through M to produce the ordered candidate multiset $\tilde{C}(q)$, and §5.3 reduces $\tilde{C}(q)$ to the bounded pool $C_B(q)$ and passes it to the stage-2 reranker R . For each operator, we also state the corresponding per-query cost.

5.1 Prefix-Constrained Key Decoding

Given a query q , the decoder D produces a bounded ordered list of candidate keys $\hat{S}_L(q)$ by traversing the trie T under beam width

L . At decoding step m , each surviving prefix $s_{<m}$ is expanded only through its admissible trie children,

$$\mathcal{V}(s_{<m}) = \text{Children}_T(s_{<m}), \quad (17)$$

so prefixes outside T are pruned before scoring. The output of this operator is therefore a bounded list of valid semantic-ID keys. The first budgeted object in stage 1 is thus the decoded key list $\hat{S}_L(q)$, whose size is controlled by L . This ordered key list is then resolved through M by the lookup operator in §5.2. Its per-query cost is

$$T_{\text{dec}} = O(c_D \cdot K \cdot L), \quad (18)$$

where K is the semantic-ID length and c_D is the per-step decoder cost.

5.2 Key-to-Video Lookup and Candidate Materialization

The lookup operator consumes the decoded key list $\hat{S}_L(q)$ and resolves each key $s \in \hat{S}_L(q)$ through the key-to-video mapping M , which serves as the stage-1 access structure from semantic-ID keys to repository videos. This yields the ordered candidate multiset

$$\tilde{C}(q) = \bigsqcup_{s \in \hat{S}_L(q)} M(s), \quad (19)$$

where $\bigsqcup$ denotes multiset concatenation in key order. The result is not yet the final candidate set: multiplicity arises because a video may be indexed under multiple keys, and a key may map to multiple videos. Unlike dense retrieval, which materializes candidates through vector similarity search over stored video embeddings, GRDR materializes candidates by resolving a bounded decoded key list through M . The resulting ordered multiset is then compacted by the deduplication operator in §5.3. Its per-query cost is

$$T_{\text{lookup}} = O(|\tilde{C}(q)|), \quad (20)$$

where $|\tilde{C}(q)|$ is the number of materialized entries before deduplication.

5.3 Candidate Deduplication and Dense Reranking

The deduplication operator consumes the candidate multiset $\tilde{C}(q)$ and produces the bounded stage-1 output

$$C_B(q) = \text{Prefix}_B(\text{Dedup}(\tilde{C}(q))), \quad |C_B(q)| \leq B. \quad (21)$$

Because $\tilde{C}(q)$ is a multiset, deduplication is a required stage-1 operator. GRDR deduplicates by first occurrence, which preserves the key-induced retrieval order while removing repeated video entries. Stage 2 then applies the dense reranker R only to $C_B(q)$ to produce the final ranking $\pi(q)$. The per-query cost is

$$T_{\text{dedup}} = O(|\tilde{C}(q)|), \quad T_{\text{rerank}} = O(B \cdot c_R), \quad (22)$$

where c_R is the reranker’s per-pair cost.

6 EXPERIMENTS

To validate our contributions and demonstrate the practical benefits of GRDR, we conduct extensive experiments to answer the following research questions (RQs): **(RQ1)** How effective is GRDR compared to existing TVR baselines? **(RQ2)** How much index storage and query-time latency does GRDR save compared to baselines

Table 1: Performance comparison on the full-corpus setting. The search pool is the union of training and test sets. The top two Generative Recall + Dense Rerank models are highlighted in bold and underlined. T_{latency} is measured in milliseconds (ms).

Method	Params	MSR-VTT [49]				ActivityNet [11]				DiDeMo [12]				LSMDC [39]			
		R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}
Search Pool Size		10,000				14,926				9,384				102,055			
Dense Retrieval																	
CLIP4CLIP [34]	151M	10.8	21.7	28.1	2456	20.5	43.4	55.6	3658	19.0	36.9	48.2	2282	4.0	7.6	10.6	24941
X-Pool [10]	153M	22.0	41.0	51.2	3795	21.5	45.7	57.9	5641	22.5	42.6	51.6	3726	5.4	10.2	12.1	38029
InternVideo2-1B [46]	1412M	24.4	43.2	52.4	2460	33.9	59.5	70.9	3661	26.0	46.3	54.5	2286	5.7	10.4	13.9	24945
Dense Recall + Dense Rerank																	
EERCF [43]	164M	20.5	42.7	50.9	4843	18.9	41.3	53.4	7628	10.4	27.2	35.3	4329	3.4	6.4	8.8	27252
ANN Recall + Dense Rerank																	
Flat	-	21.5	40.7	50.0	-	21.7	42.4	51.9	-	10.1	23.1	30.7	-	2.9	4.9	6.8	-
IVF	-	20.5	38.6	46.9	-	21.3	41.7	51.1	-	10.4	23.1	29.9	-	2.7	4.5	6.2	-
HNSW	-	21.4	40.5	49.8	-	21.7	42.3	51.9	-	10.1	23.2	30.5	-	2.9	4.9	6.5	-
Generative Recall + Dense Rerank [10]																	
TIGER [38]	69M	8.2	14.6	18.4	218	3.7	6.6	8.1	248	3.9	6.6	7.7	243	0.1	0.1	0.1	553
AVG [26]	69M	10.3	19.8	24.3	221	4.9	9.9	12.4	219	4.5	7.5	8.7	220	0.2	0.4	0.4	235
T2VIndexer [29]	144M	16.0	31.2	38.8	198	17.5	37.9	49.0	207	13.0	23.9	28.9	205	1.2	1.9	2.7	211
GRDR (Ours)	63M	17.4	32.2	39.7	184	19.2	41.1	51.8	116	15.5	29.7	36.1	119	2.1	4.8	5.9	121

Table 2: Performance comparison on standard inductive setting. The search pool is restricted to the test set. The top two Generative Recall + Dense Rerank models are highlighted in bold and underlined. T_{latency} is measured in milliseconds (ms).

Method	Params	MSR-VTT [49]				ActivityNet [11]				DiDeMo [12]				LSMDC [39]			
		R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}	R@1	R@5	R@10	T _{latency}
Search Pool Size		1,000				4,917				1,003				1,000			
Dense Retrieval																	
CLIP4CLIP [34]	151M	44.5	71.4	81.6	251	40.5	72.4	-	1211	43.4	70.2	80.6	245	22.6	41.0	49.1	243
X-Pool [10]	153M	46.9	72.8	82.2	388	35.0	65.9	78.5	1899	41.4	70.6	79.9	394	25.2	43.7	53.5	382
InternVideo2-1B [46]	1412M	45.9	70.6	78.7	255	47.4	76.0	85.2	1214	45.3	71.6	79.6	257	25.1	42.7	50.5	255
Dense Recall + Dense Rerank																	
EERCF [43]	164M	47.8	74.1	84.1	2638	43.1	74.5	86.0	5179	27.1	54.5	63.8	2292	17.0	32.1	41.1	2554
ANN Recall + Dense Rerank																	
Flat	-	47.1	72.8	82.2	-	33.9	63.2	76.1	-	28.6	55.6	65.8	-	15.0	31.3	41.5	-
IVF	-	44.7	69.5	78.1	-	33.3	61.9	74.1	-	27.3	53.8	63.9	-	15.3	30.2	38.6	-
HNSW	-	47.1	72.8	82.2	-	33.9	63.2	76.0	-	28.5	55.6	65.8	-	15.0	31.2	41.5	-
Generative Recall + Dense Rerank [10]																	
TIGER [38]	69M	36.0	55.3	62.5	245	8.9	15.9	18.3	243	17.4	28.9	32.8	242	4.6	8.3	10.1	264
AVG [26]	69M	40.4	61.4	69.9	227	14.3	26.4	30.5	237	24.9	40.5	45.3	235	11.7	18.7	21.3	234
T2VIndexer [29]	144M	42.3	61.4	69.6	179	26.0	46.4	53.9	183	34.8	53.4	60.4	179	20.6	33.7	38.2	193
GRDR (Ours)	63M	46.0	70.1	78.0	136	33.7	63.7	76.6	125	39.9	65.8	74.2	118	23.5	39.4	46.2	144

during practical corpus-scale retrieval? (RQ3) How do the main architectural components of GRDR contribute to its retrieval performance? (RQ4) How do key hyperparameters of GRDR affect its retrieval performance and query-time latency?

6.1 Experiment Setup

Datasets. We evaluate GRDR on four widely used text-to-video retrieval benchmarks. **MSR-VTT** [49] consists of 10,000 video clips from 20 categories, each annotated with 20 sentences. We utilize the standard split of 9,000 videos for training and 1,000 for testing. **ActivityNet** [11] comprises 19,000 YouTube videos across 200 activity categories, averaging 5 captions per clip. We utilize the standard partition of 10,009 training and 4,917 test samples. **LSMDC** [39] contains 118,000 short clips derived from 202 movies, providing 101,055 training and 1,000 test samples. Finally, **DiDeMo** [12] consists of 10,000 videos with 26,000 detailed moment descriptions, averaging 5 captions per clip. We utilize the split with 8,381 training and 1,003 test samples.

Evaluation Metrics. We evaluate retrieval performance using standard Recall@K metrics (R@1, R@5, R@10). We measure the average inference latency per query to evaluate retrieval efficiency. For two-stage methods, the total inference latency per query T_{latency} combines both recall and rerank stages: $T_{\text{latency}} = T_{\text{recall}} + T_{\text{rerank}}$, where in cases of a GR-based recall model, T_{recall} denotes the semantic ID generation time via beam search, including query encoding and auto-regressive decoding. We report T_{latency} by averaging across the test queries. This metric directly reflects the system’s response time in deployment scenarios.

Evaluation Settings. We evaluate all methods under two distinct settings to test different capabilities:

(i) Inductive Setting. Following the standard protocol in prior TVR work [10, 34, 43], the search pool in the inductive setting is restricted to unseen videos from the test set. This setting evaluates the model’s ability to generalize to unseen videos.

(ii) Full-Corpus Setting. The search pool contains the union of training and test videos. We introduce this setting to better reflect

real-world deployment, where systems must locate content within dynamic libraries containing both historical (seen) and newly uploaded (unseen) videos.

6.2 Baselines

Dense Retrieval Methods. We compare against representative dense retrieval baselines. **CLIP4CLIP** [34] adapts dual-encoder architecture to video retrieval with temporal pooling. **X-Pool** [10] extends this with frame-level cross-modal attention. **InternVideo2** [46] scales to 1B parameters with large-scale pretraining, achieving SOTA result in video tasks. We use it as the backbone for all GR methods in our experiments. **EERCF** [43] is the only dense method adopting the recall-and-rerank paradigm, using dense recall followed by frame-level and patch-level interaction for reranking. We integrate the official implementations of these methods into our GRDR codebase.

Generative Retrieval Methods. GR methods serve as effective candidate generators during the recall stage. **TIGER** [38] first establishes GR for sequential recommendation, demonstrating strong generative retrieval capability. **AVG** [26] extends the TIGER to image retrieval by incorporating cross-modal alignment during tokenization. The tokenizer leverages pre-trained video features to produce semantic IDs that better capture visual-textual alignment. **T2VIndexer** [29] is tailored for video retrieval by constructing hierarchical semantic IDs through semantic tree clustering. For fair comparison, we utilize the same InternVideo2-Stage2-1b [46] as video feature encoder and T5-Small [37] as the retriever, and plug in the state-of-the-art dense model X-Pool [10] to perform reranking. We configure all baseline methods with 4 codebook layers of size 256 to ensure sufficient semantic capacity.

6.3 Implementation Details

Model Architecture. We extract video features f_v using the InternVideo2-Stage2-1b [46] backbone. We configure the Multi-View Video Tokenizer with $N_o = 4$, codebook layers $M = 3$, and length $K = 128$. For LSMDC [39], we adjust these to $N_o = 1$ and $K = 200$ to accommodate its specific domain and scale. For consistency with other baselines, we employ T5-Small [37] as the generative retriever and X-Pool [10] for dense reranking.

Training Configuration. We use AdamW optimizer [33] with learning rate 1×10^{-4} and batch size 512. We employ a progressive training schedule: The first codebook layer requires 3 epochs of cross-modal pre-alignment and 4 of co-training. Subsequent layers reduce alignment to 1 epoch while maintaining 4 epochs of co-training. To provide sufficient query diversity for multi-view learning, we generate pseudo queries using GPT-5-mini [1]. Each video is augmented to around 50 queries for MSR-VTT [49], ActivityNet [11], and DiDeMo [12].

Inference Configuration. Beam size B is tuned per dataset to generate approximately 100-150 candidate videos per query, balancing recall coverage against reranking cost. Generated semantic IDs are then mapped to videos and deduplicated. During deduplication, we preserve the beam ranking order to maintain retrieval priority.

Computational Resources. All experiments are conducted on a single NVIDIA A6000 GPU (48GB). For latency measurement, we simulate real-time single-query scenarios using a batch size of 1, reporting the average over at least 100 queries after warm-up.

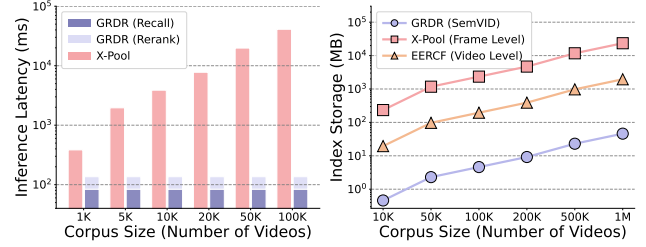


Figure 4: Efficiency Scalability Analysis. (Left) Query-time latency T_{latency} (ms) comparison between GRDR and dense retrieval across varying corpus sizes. (Right) Index storage requirements (MB) comparing GRDR’s semantic IDs against dense frame-level and video-level features.

6.4 Overall Performance (RQ1)

Inductive Setting. Table 2 shows GRDR consistently outperforms all GR baselines across four benchmarks. Since X-Pool [10] serves as our reranker, we use its standalone performance as the effectiveness ceiling. GRDR approaches this ceiling with minimal gaps of 0.9 on MSR-VTT [49]. These minimal differences indicate our recall stage successfully retrieves nearly all videos the reranker needs for accurate ranking. In contrast, TIGER [38] and AVG [26] suffer from semantic ambiguity due to one-to-one mapping. Although AVG [26] improves over TIGER [38] by 4.4 on MSR-VTT [49] through text features incorporation, it still remains 5.6 below GRDR. T2VIndexer’s [29] offline clustering without retrieval feedback produces codes misaligned with textual relevance, making its performance consistently below GRDR. EERCF [43] shows degraded performance on some dataset due to our computational constraints. Despite this, it remains valuable as the sole Dense Recall + Dense Rerank baseline for efficiency comparison.

Full-Corpus Setting. The expanded search pool amplifies scalability challenges as training videos create difficult distractors with high semantic overlap. As shown in Table 1, all methods suffer substantial degradation. However, GRDR maintains superior performance over GR baselines. Notably, TIGER [38] and AVG [26] collapse to 0.1 and 0.2 R@1 on LSMDC [39]. This almost failed performance demonstrates that semantic ambiguity becomes critical at scale, where a single semantic ID cannot disambiguate nuanced content. Interestingly, among dense retrieval methods, InternVideo2 [46] outperforms all others even on datasets where it underperformed in the Inductive Setting. This suggests that model scaling improves generalization to larger corpora.

6.5 Efficiency Analysis (RQ2)

Inference Latency Analysis. GRDR achieves 3× to 15× speedup over dense retrieval baselines in the Inductive Setting. Among GR methods, GRDR also achieves the lowest latency. TIGER [38] and AVG [26] require a codebook configuration (4 layers, size 256) to fully quantize videos into diverse entries. Moreover, their implementations use a single shared vocabulary across all layers rather than independent codebooks. This results in vocabulary redundancy that increases decoding overhead. T2VIndexer [29] builds a compact

Table 3: Ablation study of key components in GRDR. We evaluate the impact of Multi-view Tokenizer, Unified Co-training, Hierarchical Consistency Loss ($\mathcal{L}_{HC}$) and Cross-Modal Alignment Loss ($\mathcal{L}_{CL}$) on MSR-VTT [49] and ACTNET [11].

Model	MSR-VTT [49]			ActivityNet [11]		
	R@1	R@5	R@10	R@1	R@5	R@10
Inductive Setting						
GRDR (Ours)	46.0	70.1	78.0	33.6	63.6	76.5
w/o Multi-view Tokenizer	44.8 (-1.2)	67.9	77.1	33.3 (-0.3)	62.9	75.7
w/o Unified Co-training	44.5 (-1.5)	66.8	75.8	32.6 (-1.0)	62.1	74.5
w/o $\mathcal{L}_{HC}$	43.3 (-2.7)	65.8	73.9	32.7 (-0.9)	61.2	73.9
w/o $\mathcal{L}_{CL}$	38.9 (-7.1)	58.5	65.7	24.1 (-9.5)	44.8	53.9
Full-Corpus Setting						
GRDR (Ours)	17.4	32.2	39.7	19.2	41.1	51.8
w/o Multi-view Tokenizer	15.0 (-2.4)	29.0	35.6	18.5 (-0.7)	39.1	50.0
w/o Unified Co-training	15.3 (-2.1)	28.3	34.6	18.7 (-0.5)	39.3	49.9
w/o $\mathcal{L}_{HC}$	14.9 (-2.5)	28.3	34.7	17.9 (-1.3)	38.0	48.0
w/o $\mathcal{L}_{CL}$	7.2 (-10.2)	15.5	18.8	12.8 (-6.4)	26.3	33.2

codebook through k-means tree clustering, yet introduces a Prefix-aware decoder adapter that adds inference overhead. Furthermore, GRDR’s multi-view tokenization provides multiple retrieval paths per video, enabling equivalent recall coverage with fewer beam candidates. EERCf [43] incurs the highest latency on inductive setting due to its cross-modal interaction reranking, which requires on-the-fly processing of frame-level and patch-level features. We observe that GRDR exhibits a slight latency increase from 136ms to 184ms on MSR-VTT Full-Corpus Setting. This stems from code collisions between training and test sets, which generate additional candidates requiring deduplication. Despite this minor increase, the efficiency gap over dense retrieval widens dramatically at scale. As shown in the left of Figure 4, dense retrieval latency grows linearly with corpus size while GRDR remains constant, achieving up to 300× speedup on LSMDC [39] with 0.1 million videos.

Index Storage Analysis. As illustrated in Figure 4 right, GRDR achieves substantial storage reduction compared to representative dense retrieval baselines. Specifically, GRDR requires 42× less storage than video-level embeddings used by CLIP4Clip [34] and 500× less than frame-level embeddings used by X-Pool [10]. This reduction stems from replacing high-dimensional continuous embeddings with compact discrete semantic IDs. To simulate a practical large-scale scenario, we increase the GRDR codebook layers to 6 across 4 semantic views. In contrast, dense methods require storing 512-dimensional float vectors for each frame or video. Projecting to industry-scale deployment, a 1M video corpus requires only 46MB to store semantic IDs, enabling efficient retrieval on resource-constrained environments where dense embedding storage becomes prohibitive.

6.6 Effectiveness of Components (RQ3)

To validate our architectural contributions, we conduct ablation studies and qualitative analysis to examine how each component affects retrieval performance.

Effectiveness of Individual Components. We systematically remove individual modules and present results in Table 3. All components contribute meaningfully, as removing any single one leads to consistent performance degradation across both datasets and settings. **Multi-View Video Tokenizer** enables each video to be encoded into multiple semantic IDs. Replacing it with a standard

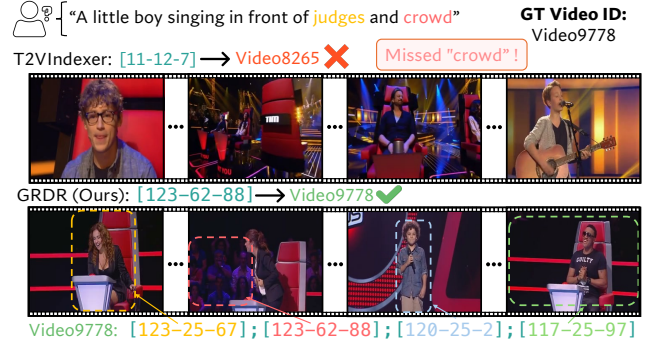


Figure 5: Case study of retrieval results. T2VIndexer [29] (Top) retrieves a video but only matching partial the subject. GRDR (Bottom) retrieves the ground-truth video containing the specific details from the query.

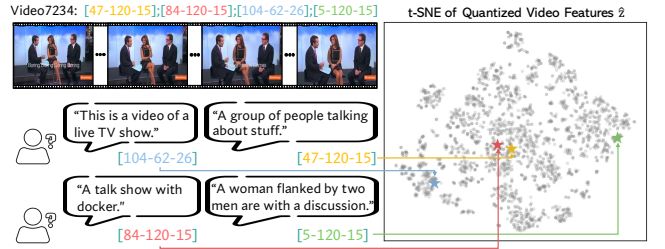


Figure 6: Visualization of our Multi-View Video Tokenizer. This t-SNE plot visualizes the quantized video features $\hat{z}$ of all semantic IDs generated from the MSR-VTT [49] test set.

single-view video tokenizer causes performance drops, with larger degradation under Full-Corpus setting. This indicates that multi-view encoding becomes increasingly important at scale, where diverse semantic access paths help distinguish similar videos. **Unified Co-Training** couples the tokenizer and retriever through a shared codebook for end-to-end optimization. Replacing it with a two-stage training pipeline degrades performance, confirming that bidirectional gradient flow allows retrieval objectives to directly refine video quantization. During unified co-training, **Hierarchical Consistency Loss ($\mathcal{L}_{HC}$)** prevents code drift during progressive training. Removal causes degradation on both settings as earlier codebook layers become unstable when later layers train. **Cross-Modal Alignment Loss ($\mathcal{L}_{CL}$)** bridges both components by aligning video latent features f_v with decoder output features h_m in a shared embedding space. Removing this loss causes the most severe degradation. Without alignment, codebook collision rates increase substantially, amplifying semantic ambiguity. This confirms that without explicit cross-modal supervision, the tokenizer produces codes optimized for visual reconstruction rather than textual relevance.

Case Study. Figure 5 presents a case on a complex query requiring multiple semantic elements. T2VIndexer [29] employs one-to-one mapping, retrieving a video matching only the main subject while missing contextual details. In contrast, GRDR encodes the

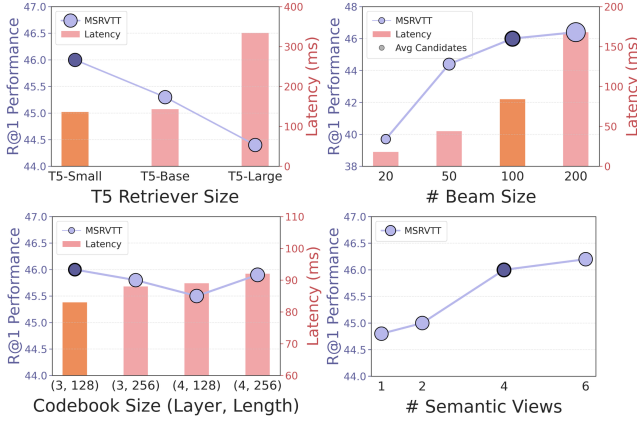


Figure 7: Hyper-parameter sensitivity on MSRVTT [49] dataset under inductive setting.

ground-truth video into four distinct semantic IDs, each capturing a different facet: ‘judges’, ‘crowd’, ‘singing boy’, and ‘show context’. When the query specifically mentions ‘crowd’, GRDR successfully retrieves the video via the corresponding semantic view. This demonstrates that multi-view encoding resolves semantic ambiguity, enabling accurate retrieval even for highly specific queries. **Qualitative Analysis of Multi-View Semantic IDs.** To further validate that our multi-view tokenizer captures genuinely distinct semantics, Figure 6 visualizes the quantized video features $\hat{z}$ via t-SNE. Four diverse queries successfully retrieve Video7234 through these different semantic IDs, which demonstrates GRDR’s ability to recall the same video from multiple semantic perspectives. This spatial distribution confirms that query-guided encoding effectively diversifies semantic representations.

6.7 Hyper-parameter Study (RQ4)

We conducted a series of experiments to investigate the impact of the four primary hyper-parameters on the trade-off between the model’s effectiveness and efficiency.

Impact of T5 Retriever backbone Size. We evaluate three T5 variants [37]: T5-Small (60M), T5-Base (220M), and T5-Large (800M). As shown in Figure 7 (top-left), increasing model size yields no performance gains while inference latency triples for T5-Large. This indicates T5-Small sufficiently captures query semantics. Our Unified Co-training drives this efficiency by aligning components in a shared semantic space, simplifying semantic ID prediction without requiring larger model capacity.

Number of Beams Size. We vary beam size from 20 to 200, with circle size indicating average candidates generated. Performance improves rapidly initially but plateaus after 100 beams, while latency continues growing. We select beam size 100 as the optimal balance between recall coverage and reranking cost.

Impact of Codebook Size. We analyze codebook dimensions (layers and length) in Figure 7 (bottom-left). Increasing dimensions yields no significant gains, indicating that Multi-View semantic IDs and Unified Co-training effectively capture complex video semantics even with highly compressed code space. Since latency

increases with codebook size, we select configuration (3, 128) to maximize both efficiency and effectiveness.

Number of Semantic Views. We vary N_v from 1 to 6. Performance improves initially but saturates beyond 4 views. Too few semantic IDs cause semantic ambiguity by failing to capture diverse user intents, while excessive views result in redundancy and increased semantic ID collisions. We identify $N_v = 4$ as optimal configuration.

7 RELATED WORK

Text-to-Video Retrieval. Text-to-Video Retrieval (TVR) aims to retrieve videos from a corpus based on their semantic relevance to a text query. Existing methods fall into two paradigms distinguished by their cross-modal interaction strategy. Cross-Encoder Retrieval [20, 24, 46] leverages deep cross-modal fusion to achieve high retrieval accuracy. With recent advances in large-scale Vision-Language Models (VLMs) [2–4, 9, 23, 44, 51], models such as InternVideo series [46–48] and BLiM [20] employ cross-attention mechanisms to capture fine-grained interactions. However, this architecture requires a full forward pass for every query-video pair at retrieval time, making it impractical for large-scale retrieval. The Dense Retrieval paradigm addresses this bottleneck by decoupling video and text encoding into separate branches. Videos can be encoded offline, reducing retrieval to nearest-neighbor search over precomputed embeddings. Building on pretrained vision-language models such as CLIP [36], subsequent methods [7, 10, 17, 18, 22, 25, 31, 34, 50, 55–57] extend this paradigm through temporal modeling and cross-modal alignment mechanisms. Recently, InternVideo2 [46] proposes a CLIP-style dual-encoder variant to further improve dense retrieval baselines through model scaling. Despite these advances, Dense Retrieval faces inherent scalability limitations. Storage complexity grows linearly with corpus size due to high-dimensional video features. Inference complexity also scales linearly, as retrieval requires exhaustive similarity computation across all candidates. EERCF [43] attempts to mitigate inference costs through Coarse-to-Fine Retrieval. It proposes a two-stage framework that first filters candidates before fine-grained reranking. However, the recall stage still relies on dense representations, inheriting the same storage overhead. These limitations motivate Generative Retrieval, which achieves compact storage and efficient inference through the generation of discrete semantic IDs.

Generative Retrieval. Generative Retrieval (GR) [8, 13, 16, 26, 29, 30, 32, 38, 40–42, 45] reformulates retrieval as a sequence-to-sequence generation task, where models directly produce target identifiers conditioned on queries. Following its foundational success in document retrieval [16, 28, 41, 42, 45], the paradigm has recently been adapted to recommendation systems [8, 13, 30, 32, 38, 40]. TIGER [38] first proposes semantic IDs using Residual Quantization (RQ) [21] to generate hierarchical discrete codes for items in sequential recommendation. In addition, Actionpiece [13] extends this by tokenizing user behaviors using a variable-length mechanism. ETEGRec [30] further advanced this direction by jointly optimizing item tokenization and recommendation objectives, eliminating the disjoint multi-stage pipeline. Building upon these developments, GR has been extended to multi-modal tasks [19, 53]. GRACE [27] introduces structured natural language identifiers to enable generative cross-modal retrieval within MLLMs [48]. IRGen [54] employs discrete visual tokens for image retrieval, while AVG

[26] introduces cross-modal alignment during tokenization. Video retrieval presents unique challenges due to the semantic richness of videos. T2VIndexer [29] extends GR to video retrieval by constructing hierarchical video identifiers through offline clustering. However, videos are inherently polysemous, yet these approaches assign each video a single semantic ID. This one-to-one mapping forces diverse query intents to converge on identical codes.

8 CONCLUSION

We present GRDR, a Generative Recall and Dense Reranking framework that addresses two fundamental challenges limiting generative retrieval for TVR: (i) semantic ambiguity, where polysemous videos are forced into single semantic IDs, and (ii) cross-modal misalignment, where semantic IDs lack supervision from text queries. To resolve these challenges, GRDR introduces a Multi-View Video Tokenizer that assigns multiple semantic IDs per video via query-guided contrastive learning, and a Unified Co-Training scheme that couples tokenizer and retriever through a shared codebook for end-to-end optimization. Experiments on four benchmarks demonstrate that GRDR achieves competitive accuracy with dense retrievers while reducing storage by an order of magnitude and accelerating inference by up to 300×. Our established GRDR provides an effective generative retrieval recall stage that bridges the gap between scalability and accuracy, paving the way for future research in efficient text-to-video retrieval.

ACKNOWLEDGMENTS

This work was supported by the [...] Research Fund of [...] (Number [...]). Additional funding was provided by [...] and [...]. We also thank [...] for contributing [...].

REFERENCES

- [1] [n.d.]. GPT-5 mini Model | OpenAI API. <https://platform.openai.com>
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *CoRR* abs/2502.13923 (2025). <https://doi.org/10.48550/ARXIV.2502.13923> arXiv:2502.13923
- [3] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. 2023. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. *CoRR* abs/2312.14238 (2023). <https://doi.org/10.48550/ARXIV.2312.14238> arXiv:2312.14238
- [4] Zhi Chen, Zecheng Zhao, Jingcai Guo, Jingjing Li, and Zi Huang. 2025. SVIP: Semantically Contextualized Visual Patches for Zero-Shot Learning. *CoRR* abs/2503.10252 (2025).
- [5] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *RecSys*. ACM, 191–198.
- [6] Zuoqiao Dai, Fangtao Shao, Qingkun Su, Zilong Dong, and Siyu Zhu. 2023. Fine-grained Text-Video Retrieval with Frozen Image Encoders. *arXiv preprint arXiv:2307.09972* (2023).
- [7] Chaorui Deng, Qi Chen, Pengda Qin, Da Chen, and Qi Wu. 2023. Prompt Switch: Efficient CLIP Adaptation for Text-Video Retrieval. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 15602–15612. <https://doi.org/10.1109/ICCV51070.2023.01434>
- [8] Yijie Ding, Yupeng Hou, Jiacheng Li, and Julian J. McAuley. 2024. Inductive Generative Recommendation via Retrieval-based Speculation. *CoRR* abs/2410.02939 (2024). <https://doi.org/10.48550/ARXIV.2410.02939> arXiv:2410.02939
- [9] Yuxia Fu, Zhizhen Zhang, Yuqi Zhang, Zijian Wang, Zi Huang, and Yadan Luo. 2025. MergeVLA: Cross-Skill Model Merging Toward a Generalist Vision-Language-Action Agent. *CoRR* abs/2511.18810 (2025).
- [10] Satya Krishna Gorti, Noël Vouitsis, Junwei Ma, Keyvan Golestan, Maksims Volkovs, Animesh Garg, and Guangwei Yu. 2022. X-Pool: Cross-Modal Language-Video Attention for Text-Video Retrieval. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 4996–5005. <https://doi.org/10.1109/CVPR52688.2022.00495>
- [11] Fabian Caba Heilbron, Victor Escorcia, Bernard Ghanem, and Juan Carlos Nieves. 2015. ActivityNet: A large-scale video benchmark for human activity understanding. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*. IEEE Computer Society, 961–970. <https://doi.org/10.1109/CVPR.2015.7298698>
- [12] Lisa Anne Hendricks, Oliver Wang, Eli Shechtman, Josef Sivic, Trevor Darrell, and Bryan C. Russell. 2017. Localizing Moments in Video with Natural Language. In *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*. IEEE Computer Society, 5804–5813. <https://doi.org/10.1109/ICCV.2017.618>
- [13] Yupeng Hou, Jianmo Ni, Zhankui He, Naveen Sachdeva, Wang-Cheng Kang, Ed H. Chi, Julian J. McAuley, and Derek Zhiyuan Cheng. 2025. ActionPiece: Contextually Tokenizing Action Sequences for Generative Recommendation. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. OpenReview.net. <https://openreview.net/forum?id=h2oNQOzbc5>
- [14] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *KDD*. ACM, 2553–2561.
- [15] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [16] Bowen Jin, Hansi Zeng, Guoyin Wang, Xiusi Chen, Tianxin Wei, Ruirui Li, Zhengyang Wang, Zheng Li, Yang Li, Hanqing Lu, Suhang Wang, Jiawei Han, and Xianfeng Tang. 2024. Language Models as Semantic Indexers. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=sYeioWoF9u>
- [17] Peng Jin, Hao Li, Zesen Cheng, Kehan Li, Xiangyang Ji, Chang Liu, Li Yuan, and Jie Chen. 2023. DiffusionRet: Generative Text-Video Retrieval with Diffusion Model. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 2470–2481. <https://doi.org/10.1109/ICCV51070.2023.00234>
- [18] Xiaojie Jin, Bowen Zhang, Weibo Gong, Kai Xu, Xueqing Deng, Peng Wang, Zhao Zhang, Xiaohui Shen, and Jiashi Feng. 2024. MV-Adapter: Multimodal Video Transfer Learning for Video Text Retrieval. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 27134–27143. <https://doi.org/10.1109/CVPR52733.2024.02563>
- [19] Sungyeon Kim, Xinliang Zhu, Xiaofan Lin, Muhammet Bastan, Douglas Gray, and Suha Kwak. 2025. GENIUS: A Generative Framework for Universal Multimodal Search. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*. Computer Vision Foundation / IEEE, 19659–19669. <https://doi.org/10.1109/CVPR52734.2025.01831>
- [20] Dohwan Ko, Ji Soo Lee, Minhyuk Choi, Zihang Meng, and Hyunwoo J. Kim. 2025. Bidirectional Likelihood Estimation with Multi-Modal Large Language Models for Text-Video Retrieval. *CoRR* abs/2507.23284 (2025). <https://doi.org/10.48550/ARXIV.2507.23284> arXiv:2507.23284
- [21] Doyup Lee, Chihyeon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive Image Generation using Residual Quantization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 11513–11522. <https://doi.org/10.1109/CVPR52688.2022.01123>
- [22] Jie Lei, Linjie Li, Luowei Zhou, Zhe Gan, Tamara L. Berg, Mohit Bansal, and Jingjing Liu. 2021. Less Is More: ClipBERT for Video-and-Language Learning via Sparse Sampling. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 7331–7341. <https://doi.org/10.1109/CVPR46437.2021.00725>
- [23] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. 2025. LLaVA-OneVision: Easy Visual Task Transfer. *Trans. Mach. Learn. Res.* 2025 (2025). <https://openreview.net/forum?id=zKv8qULV6n>
- [24] Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. 2022. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.), Vol. 162. PMLR, 12888–12900. <https://proceedings.mlr.press/v162/li22n.html>
- [25] Pandeng Li, Chen-Wei Xie, Liming Zhao, Hongtao Xie, Jiannan Ge, Yun Zheng, Deli Zhao, and Yongdong Zhang. 2023. Progressive Spatio-Temporal Prototype Matching for Text-Video Retrieval. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. IEEE, 4077–4087. <https://doi.org/10.1109/ICCV51070.2023.00379>
- [26] Yongqi Li, Hongru Cai, Wenjie Wang, Leigang Qu, Yinwei Wei, Wenjie Li, Liqiang Nie, and Tat-Seng Chua. 2025. Revolutionizing Text-to-Image Retrieval as Autoregressive Token-to-Token Generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*,

- SIGIR 2025, Padua, Italy, July 13-18, 2025, Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.). ACM, 813–822. <https://doi.org/10.1145/3726302.3730077>
- [27] Yongqi Li, Wenjie Wang, Leigang Qu, Liqiang Nie, Wenjie Li, and Tat-Seng Chua. 2024. Generative Cross-Modal Retrieval: Memorizing Images in Multimodal Language Models for Retrieval and Beyond. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11–16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 11851–11861. <https://doi.org/10.18653/V1/2024.ACL-LONG.639>
- [28] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Multiview Identifiers Enhanced Generative Retrieval. In *ACL (1)*. Association for Computational Linguistics, 6636–6648.
- [29] Yili Li, Jing Yu, Keke Gai, Bang Liu, Gang Xiong, and Qi Wu. 2024. T2VIndexer: A Generative Video Indexer for Efficient Text-Video Retrieval. In *Proceedings of the 32nd ACM International Conference on Multimedia, MM 2024, Melbourne, VIC, Australia, 28 October 2024 - 1 November 2024*, Jianfei Cai, Mohan S. Kankanhalli, Balakrishnan Prabhakaran, Susanne Boll, Ramanathan Subramanian, Liang Zheng, Vivek K. Singh, Pablo César, Lexing Xie, and Dong Xu (Eds.). ACM, 3955–3963. <https://doi.org/10.1145/3664647.3680673>
- [30] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative Recommender with End-to-End Learnable Item Tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13–18, 2025*, Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.). ACM, 729–739. <https://doi.org/10.1145/3726302.3729989>
- [31] Yuqi Liu, Pengfei Xiong, Luhui Xu, Shengming Cao, and Qin Jin. 2022. TS2-Net: Token Shift and Selection Transformer for Text-Video Retrieval. In *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XIV (Lecture Notes in Computer Science)*, Shai Avidan, Gabriel J. Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner (Eds.), Vol. 13674. Springer, 319–335. https://doi.org/10.1007/978-3-031-19781-9_19
- [32] Zihan Liu, Yupeng Hou, and Julian J. McAuley. 2024. Multi-Behavior Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21–25, 2024*, Edoardo Serra and Francesca Spezzano (Eds.). ACM, 1575–1585. <https://doi.org/10.1145/3627673.3679730>
- [33] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *ICLR (Poster)*. OpenReview.net.
- [34] Huaishao Luo, Lei Ji, Ming Zhong, Yang Chen, Wen Lei, Nan Duan, and Tianrui Li. 2022. CLIP4Clip: An empirical study of CLIP for end to end video clip retrieval and captioning. *Neurocomputing* 508 (2022), 293–304. <https://doi.org/10.1016/j.neucom.2022.07.028>
- [35] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [36] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18–24 July 2021, Virtual Event (Proceedings of Machine Learning Research)*, Marina Meila and Tong Zhang (Eds.), Vol. 139. PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [37] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21 (2020), 140:1–140:67. <https://jmlr.org/papers/v21/20-074.html>
- [38] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Mahesh Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/20dcab0f14046a5c6b02b61da9f13229-Abstract-Conference.html
- [39] Anna Rohrbach, Atousa Torabi, Marcus Rohrbach, Niket Tandon, Christopher Joseph Pal, Hugo Larochelle, Aaron C. Courville, and Berni Schiele. 2017. Movie Description. *Int. J. Comput. Vis.* 123, 1 (2017), 94–120. <https://doi.org/10.1007/S11263-016-0987-1>
- [40] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, and Jun Xu. 2023. Generative Retrieval with Semantic Tree-Structured Item Identifiers via Contrastive Learning. *CoRR* abs/2309.13375 (2023). <https://doi.org/10.48550/ARXIV.2309.13375> arXiv:2309.13375
- [41] Weiwei Sun, Lingyong Yan, Zheng Chen, Shuaiqiang Wang, Haichao Zhu, Pengjie Ren, Zhumin Chen, Dawei Yin, Maarten de Rijke, and Zhaochun Ren. 2023. Learning to Tokenize for Generative Retrieval. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/91228b942a4528cdac031c1b68b127e8-Abstract-Conference.html
- [42] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Prakash Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. 2022. Transformer Memory as a Differentiable Search Index. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/892840a6123b5ec99ebaab8be1530fba-Abstract-Conference.html
- [43] Kaibin Tian, Yanhua Cheng, Yi Liu, Xinglin Hou, Quan Chen, and Han Li. 2024. Towards Efficient and Effective Text-to-Video Retrieval with Coarse-to-Fine Visual Representation Learning. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20–27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriaram Natarajan (Eds.). AAAI Press, 5207–5214. <https://doi.org/10.1609/AAAI.V38I6.28327>
- [44] Tianqi Wang, Jingcai Guo, Depeng Li, and Zhi Chen. 2025. On the Discrimination and Consistency for Exemplar-Free Class Incremental Learning. In *IJCAI ijcai.org*, 6424–6432.
- [45] Yuying Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Qi Chen, Yucheng Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, Xing Xie, Hao Sun, Weiwei Deng, Qi Zhang, and Mao Yang. 2022. A Neural Corpus Indexer for Document Retrieval. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/a46156db3579c3b268108ea6aca71d13-Abstract-Conference.html
- [46] Yi Wang, Kunchang Li, Xinhao Li, Jiashuo Yu, Yanan He, Guo Chen, Baoqi Pei, Rongkun Zheng, Zun Wang, Yansong Shi, Tianxiang Jiang, Songze Li, Jilan Xu, Hongjie Zhang, Yifei Huang, Yu Qiao, Yali Wang, and Limin Wang. 2024. InternVideo2: Scaling Foundation Models for Multimodal Video Understanding. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXXXV (Lecture Notes in Computer Science)*, Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.), Vol. 15143. Springer, 396–416. https://doi.org/10.1007/978-3-031-73013-9_23
- [47] Yi Wang, Kunchang Li, Yizhuo Li, Yanan He, Bingkun Huang, Zhiyu Zhao, Hongjie Zhang, Jilan Xu, Yi Liu, Zun Wang, Sen Xing, Guo Chen, Junting Pan, Jiashuo Yu, Yali Wang, Limin Wang, and Yu Qiao. 2022. InternVideo: General Video Foundation Models via Generative and Discriminative Learning. *CoRR* abs/2212.03191 (2022). <https://doi.org/10.48550/ARXIV.2212.03191> arXiv:2212.03191
- [48] Yi Wang, Xinhao Li, Ziang Yan, Yanan He, Jiashuo Yu, Xiangyu Zeng, Chenting Wang, Changlian Ma, Haian Huang, Jianfei Gao, Min Dou, Kai Chen, Wenhai Wang, Yu Qiao, Yali Wang, and Limin Wang. 2025. InternVideo2.5: Empowering Video MLLMs with Long and Rich Context Modeling. *CoRR* abs/2501.12386 (2025). <https://doi.org/10.48550/ARXIV.2501.12386> arXiv:2501.12386
- [49] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. 2016. MSR-VTT: A Large Video Description Dataset for Bridging Video and Language. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27–30, 2016*. IEEE Computer Society, 5288–5296. <https://doi.org/10.1109/CVPR.2016.571>
- [50] Hongwei Xue, Yuchong Sun, Bei Liu, Jianlong Fu, Ruihua Song, Houqiang Li, and Jiebo Luo. 2022. CLIP-ViP: Adapting Pre-trained Image-Text Model to Video-Language Representation Alignment. *CoRR* abs/2209.06430 (2022). <https://doi.org/10.48550/ARXIV.2209.06430> arXiv:2209.06430
- [51] Le Xue, Manli Shu, Anas Awadalla, Jun Wang, An Yan, Senthil Purushwalkam, Honglu Zhou, Viraj Prabhu, Yutong Dai, Michael S. Ryoo, Shrikant Kendre, Jieyu Zhang, Can Qin, Shu Zhang, Chia-Chih Chen, Ning Yu, Juntao Tan, Tulika Manoj Awalganekar, Shelby Heinecke, Huan Wang, Yejin Choi, Ludwig Schmidt, Zeyuan Chen, Silvio Savarese, Juan Carlos Niebles, Caiming Xiong, and Ran Xu. 2024. xGen-MM (BLIP-3): A Family of Open Large Multimodal Models. *CoRR* abs/2408.08872 (2024). <https://doi.org/10.48550/ARXIV.2408.08872> arXiv:2408.08872
- [52] Hamed Zamani, Michael Bendersky, Donald Metzler, Honglei Zhuang, and Xuanhui Wang. 2022. Stochastic Retrieval-Conditioned Reranking. In *ICTIR '22: The 2022 ACM SIGIR International Conference on the Theory of Information Retrieval, Madrid, Spain, July 11 - 12, 2022*, Fabio Crestani, Gabriella Pasi, and Eric Gaussier (Eds.). ACM, 81–91. <https://doi.org/10.1145/3539813.3545141>

- [53] Jianyang Zhai, Zi-Feng Mai, Chang-Dong Wang, Feidiao Yang, Xiwu Zheng, Hui Li, and Yonghong Tian. 2025. Multimodal Quantitative Language for Generative Recommendation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=v7YrljpkTF>
- [54] Yidan Zhang, Ting Zhang, Dong Chen, Yujing Wang, Qi Chen, Xing Xie, Hao Sun, Weiwei Deng, Qi Zhang, Fan Yang, Mao Yang, Qingmin Liao, Jingdong Wang, and Baining Guo. 2024. IRGen: Generative Modeling for Image Retrieval. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XV (Lecture Notes in Computer Science)*, Ales Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.), Vol. 15073. Springer, 21–41. https://doi.org/10.1007/978-3-031-72633-0_2
- [55] Zhizhen Zhang, Lei Zhu, Zhen Fang, Zi Huang, and Yadan Luo. 2025. Provable Ordering and Continuity in Vision-Language Pretraining for Generalizable Embodied Agents. *CoRR* abs/2502.01218 (2025).
- [56] Zecheng Zhao, Zhi Chen, Zi Huang, Shazia Sadiq, and Tong Chen. 2025. Continual Text-to-Video Retrieval with Frame Fusion and Task-Aware Routing. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13-18, 2025*, Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.). ACM, 1011–1021. <https://doi.org/10.1145/3726302.3729936>
- [57] Zecheng Zhao, Selena Song, Tong Chen, Zhi Chen, Shazia Sadiq, and Yadan Luo. 2025. Are Synthetic Videos Useful? A Benchmark for Retrieval-Centric Evaluation of Synthetic Videos. *CoRR* abs/2507.02316 (2025). <https://doi.org/10.48550/ARXIV.2507.02316> arXiv:2507.02316